



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №8
із дисципліни *«Технології розроблення програмного забезпечення»*
Патерни проектування
Аудіоредактор

Виконала
студентка групи ІА–34
Кузьменко В. С.

Перевірів
викладач
Мягкий М. Ю.

ЗМІСТ

Мета	3
Завдання.....	3
Тема роботи	3
Теоретичні відомості	4
Хід роботи	6
Шаблон Composite	6
Графічний інтерфейс користувача.....	10
Код системи	12
Висновок.....	15
Відповіді на теоретичні питання.....	17

Тема: Патерни проектування.

Мета: Вивчити структуру шаблонів «Composite», «Flyweight» (Пристосуванець), «Interpreter», «Visitor» та навчитися застосовувати їх в реалізації програмної системи.

Завдання:

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Тема роботи

Аудіо редактор (singleton, adapter, observer, mediator, composite, client-server)
Аудіо редактор повинен володіти наступним функціоналом: представлення аудіо даних будь-якого формату в WAVE-формі, вибір і подальші операції копіювання / вставки / вирізання / деформації по сегменту аудіозапису, можливість роботи з декількома звуковими доріжками, кодування в найбільш поширених форматах (ogg, flac, mp3).

Теоретичні відомості

Шаблони проектування для роботи з базами даних допомагають створювати масштабовані та ефективні системи, особливо для програм з великою кількістю користувачів. Вони дозволяють зменшити складність і підвищити продуктивність при зростанні обсягів даних.

Active Record: Шаблон передбачає, що кожен об'єкт відповідає за дані і поведінку, що стосуються одного рядка в базі даних. Усі операції з базою даних і логіка обробки даних розміщуються прямо в класі сутності. Active Record підходить для простих систем, таких як Ruby on Rails, але з ростом складності часто зручно перенести логіку в окремі спеціалізовані об'єкти.

Table Data Gateway: Шаблон визначає окремий клас для роботи з кожним типом даних. У цьому класі реалізовані всі запити до бази даних, включаючи збереження та видалення. Це забезпечує більшу гнучкість, легкість у тестуванні та розділення логіки роботи з даними. Для зменшення дублювання коду загальні методи можуть бути винесені в базовий клас, що також називається «репозиторієм».

Data Mapper: Шаблон, який відокремлює об'єкти програми від структури бази даних. Data Mapper забезпечує коректне відображення властивостей об'єкта до стовпців таблиць бази даних, вирішуючи невідповідності між типами даних. Це полегшує інтеграцію складних систем і дає змогу краще відображати складні об'єкти на рівні бази даних.

Шаблони роботи з об'єктами

Composite: Шаблон дозволяє створити деревоподібну структуру для представлення ієрархії "частина-ціле". Він дозволяє однаково обробляти як окремі об'єкти, так і складені структури. Наприклад, у формі можуть бути поля для введення, написи, та інші елементи, які обробляються однаково. Composite

ефективний для роботи з ієрархічними структурами, такими як замовлення, що складаються з різних продуктів і контейнерів.

Flyweight: Шаблон, який зменшує кількість об'єктів у програмі шляхом спільного використання однакових даних між різними частинами програми. Наприклад, у текстовому редакторі кожна буква може здаватися окремим об'єктом, але це насправді спільний об'єкт, на який посилаються різні частини програми. Flyweight дозволяє розділяти дані на внутрішній (загальний для всіх об'єктів) і зовнішній (унікальний для конкретного контексту).

Interpreter: Шаблон, який використовується для обробки мов і граматик. Він створює синтаксичне дерево, де кожен елемент дерева (термінальний або нетермінальний) інтерпретується в своєму контексті. Інтерпретатор підходить для малих мов або простих контекстів, однак у разі частих змін граматики може стати складним для підтримки.

Visitor: Шаблон, який дозволяє виконувати операції над об'єктами без зміни їхньої структури. Для додавання нових операцій створюються окремі класи-відвідувачі. Наприклад, для експорту графу даних у формат XML можна створити окремий клас, який виконує потрібну операцію. Однак додавання нових елементів у систему вимагатиме змін у всіх наявних класах-відвідувачах, що може ускладнити підтримку.

Використання шаблонів для роботи з базами даних і об'єктами дає змогу ефективно організувати складні системи. Вони підвищують гнучкість, продуктивність і полегшують підтримку, що особливо важливо для корпоративних додатків, які мають високі вимоги до масштабованості та навантаження.

Хід роботи

1. Ознайомитись з теоретичними відомостями щодо шаблонів проектування та їхнього використання в програмуванні.
2. Реалізувати частину функціоналу програми за допомогою класів та їх взаємодії для досягнення конкретних функціональних можливостей.
3. Застосувати один з розглянутих шаблонів проектування для реалізації частини програми.
4. Реалізувати не менше трьох класів згідно з обраною темою.
5. Підготувати звіт, що містить діаграму класів для використаного шаблону та фрагменти коду, які демонструють застосування шаблону.

Шаблон Composite

Ознайомившись з теоретичними матеріалами, було вирішено застосувати шаблон *Composite* (Компонувальник), який є структурним патерном проектування. Цей шаблон дозволяє згрупувати кілька об'єктів у єдину деревоподібну структуру, при цьому забезпечуючи спільне оброблення як окремого об'єкта, так і складеного набору об'єктів. Завдяки цьому, патерн *Composite* дозволяє клієнтському коду взаємодіяти з одиничними елементами і складеними групами однаково, без необхідності змінювати обробку кожного об'єкта окремо.

На графічному зображенні (Рис. 1 - Загальна структура шаблону *Composite*) показано, як елементи графічного інтерфейсу організовуються у вигляді дерева, де кожен вузол (елемент) може бути окремим об'єктом або складною групою елементів, що дозволяє обробляти їх як єдину структуру.

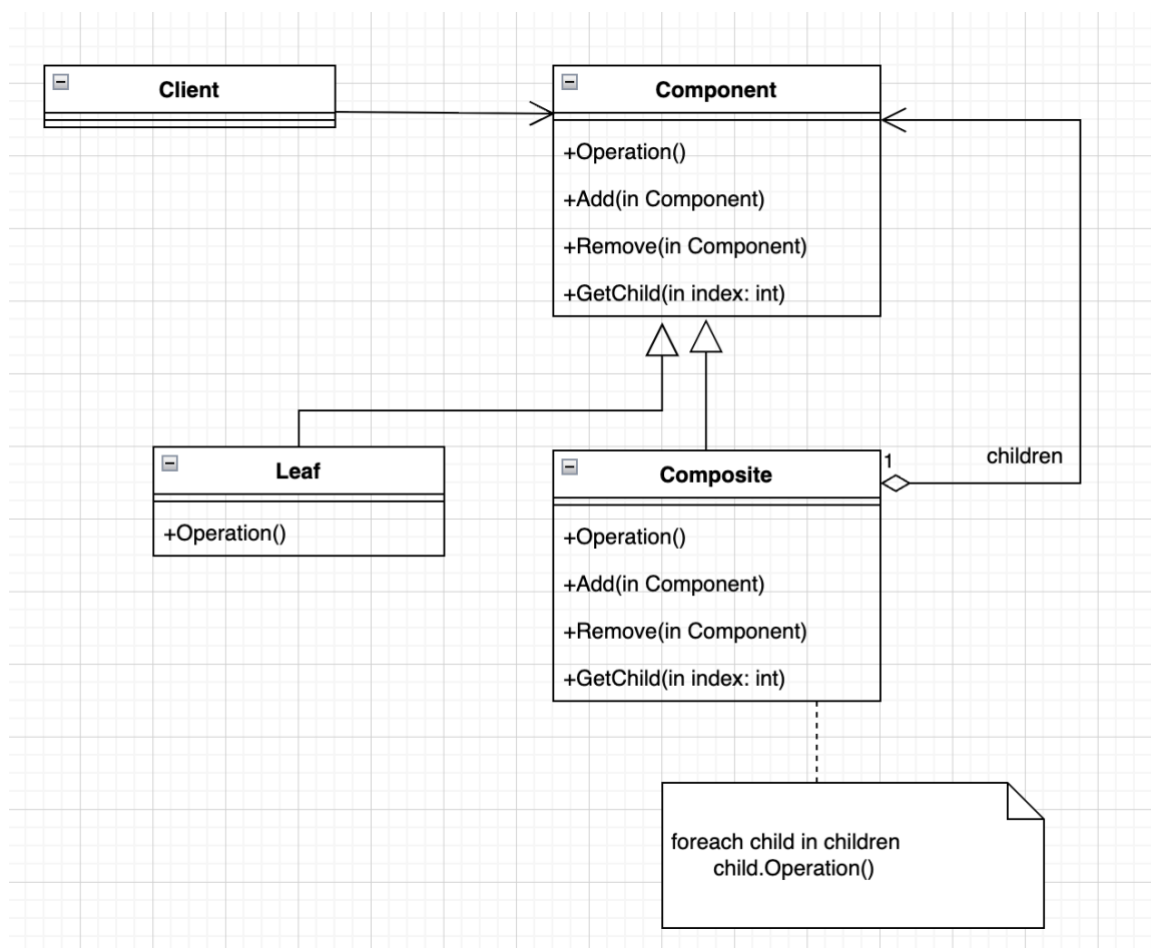


Рис. 1 - Загальна структура шаблону *Composite*

Компонент визначає загальний інтерфейс як для простих, так і для складених елементів дерева. Лист - це базовий компонент дерева, який не має підкомпонентів. Класи листя містять основну логіку, оскільки вони не передають виконання іншим елементам. Контейнер (або композит) - це складений компонент дерева, який може містити кілька дочірніх елементів, але не залежить від їхніх типів. Клієнт взаємодіє з усіма елементами дерева через єдиний інтерфейс, що забезпечує узагальнену обробку всіх компонентів.

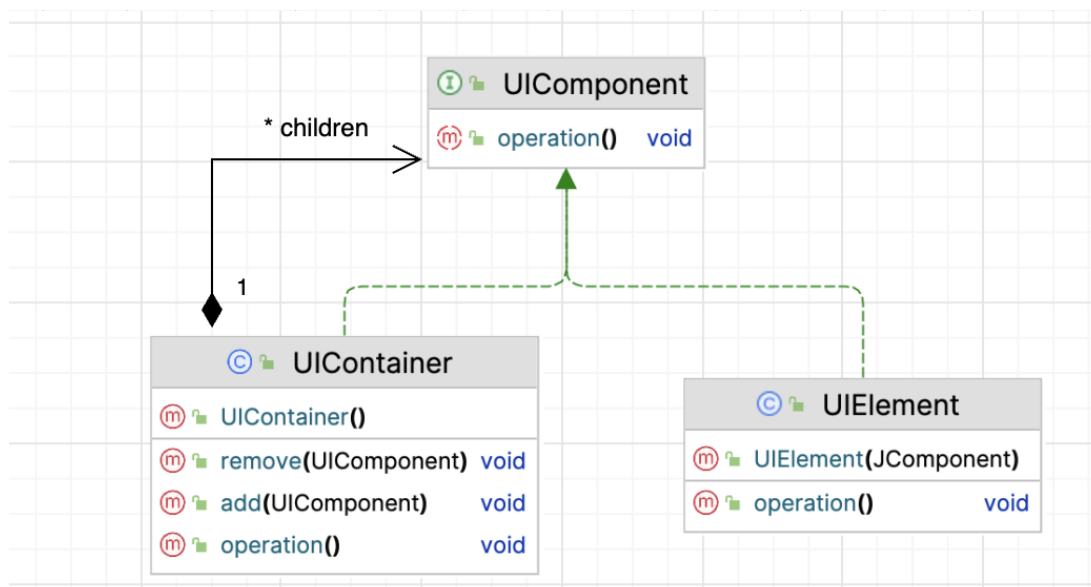


Рис. 2 - Структура реалізованого шаблону *Composite*

На Рис. 2 представлено структуру реалізованого шаблону *Composite*. У цій структурі виділено три основні компоненти: інтерфейс *UIComponent*, клас *UIContainer* (композит, або контейнер) і клас *UIElement* (лист, або окремий елемент).

Інтерфейс *UIComponent* є спільною основою для всіх елементів користувацького інтерфейсу. Він визначає метод *operation()*, який має виконувати дію, притаманну кожному конкретному компоненту. Цей метод є абстрактним і не має реалізації - кожен підклас реалізує його по-своєму. Таким чином, *UIComponent* задає єдиний контракт для всіх компонентів, що дозволяє працювати з ними через спільний тип, не знаючи, чи це контейнер, чи окремий елемент.

Клас *UIContainer* реалізує інтерфейс *UIComponent* і виступає контейнером для інших компонентів. Він має колекцію *children*, у якій зберігаються об'єкти типу *UIComponent*. Це можуть бути як інші контейнери (*UIContainer*), так і звичайні елементи (*UIElement*). На діаграмі показано композиційний зв'язок - чорний ромб біля *UIContainer* означає, що він володіє своїми дочірніми елементами, тобто якщо контейнер буде знищено, його діти також перестануть існувати.

Контейнер надає методи:

- `add(UIComponent component)` - додає новий дочірній елемент до списку;
- `remove(UIComponent component)` - видаляє вказаний дочірній елемент;
- `operation()` - викликає метод *operation()* для кожного компонента зі списку *children*, тобто передає виконання дії всім вкладеним елементам.

Таким чином, *UIContainer* може містити інші контейнери або елементи, створюючи деревоподібну структуру інтерфейсу, де кожен вузол може виконувати спільну дію.

Клас *UIElement* також реалізує інтерфейс *UIComponent*, але на відміну від контейнера, не має дочірніх елементів. Він представляє окремий елемент інтерфейсу користувача, наприклад кнопку, поле введення чи напис. *UIElement* містить поле *component*, яке зберігає графічний елемент типу *JComponent* із бібліотеки *Swing*. Конструктор *UIElement(JComponent)* ініціалізує цей компонент. Метод *operation()* у класі *UIElement* виконує дію над конкретним графічним об'єктом - наприклад, викликає його метод *repaint()*, щоб оновити відображення або змінити вигляд елемента. Таким чином, *UIElement* - це кінцевий вузол дерева, який реалізує конкретну поведінку, не містячи інших елементів.

Взаємозв'язки між класами:

- *UIComponent* - інтерфейс, який реалізують обидва класи (*UIContainer* і *UIElement*);
- *UIContainer* має список дочірніх елементів типу *UIComponent*, утворюючи зв'язок «один до багатьох»;
- *UIElement* не має дітей і є кінцевим об'єктом у структурі;
- Зв'язок між контейнером і дітьми позначено як композицію, оскільки діти залежать від існування контейнера.

Логіка роботи системи:

1. Програма створює окремі елементи (*UIElement*) і контейнери (*UIContainer*).
2. За допомогою методів *add()* контейнери наповнюються іншими елементами або навіть цілими підконтейнерами.
3. Коли викликається метод *operation()* у кореневого контейнера, він послідовно викликає цей же метод у всіх своїх дочірніх елементах.
4. Якщо серед дочірніх елементів є інші контейнери, вони, у свою чергу, виконують операцію для своїх дітей.
5. У результаті створюється рекурсивне виконання, де одна команда впливає на всю структуру незалежно від її глибини.

Переваги реалізації:

- Єдиний інтерфейс для всіх елементів UI.
- Можливість створювати складні ієрархії з будь-якою глибиною вкладеності.
- Легке додавання нових типів компонентів без зміни існуючого коду.
- Зручність рекурсивного оброблення всіх елементів через виклик одного методу *operation()*.

Отже, діаграма демонструє класичну реалізацію патерну *Composite*, де *UIComponent* задає спільний контракт, *UIContainer* керує групою підлеглих елементів, а *UIElement* реалізує конкретну поведінку окремого графічного об'єкта. Разом вони створюють гнучку та масштабовану систему для побудови графічного інтерфейсу користувача.

Графічний інтерфейс користувача

На Рис. 3 зображено головне вікно застосунку "AudioEditor" (Client), яке є прикладом використання шаблону *Composite* (Композиція) у графічному інтерфейсі користувача.

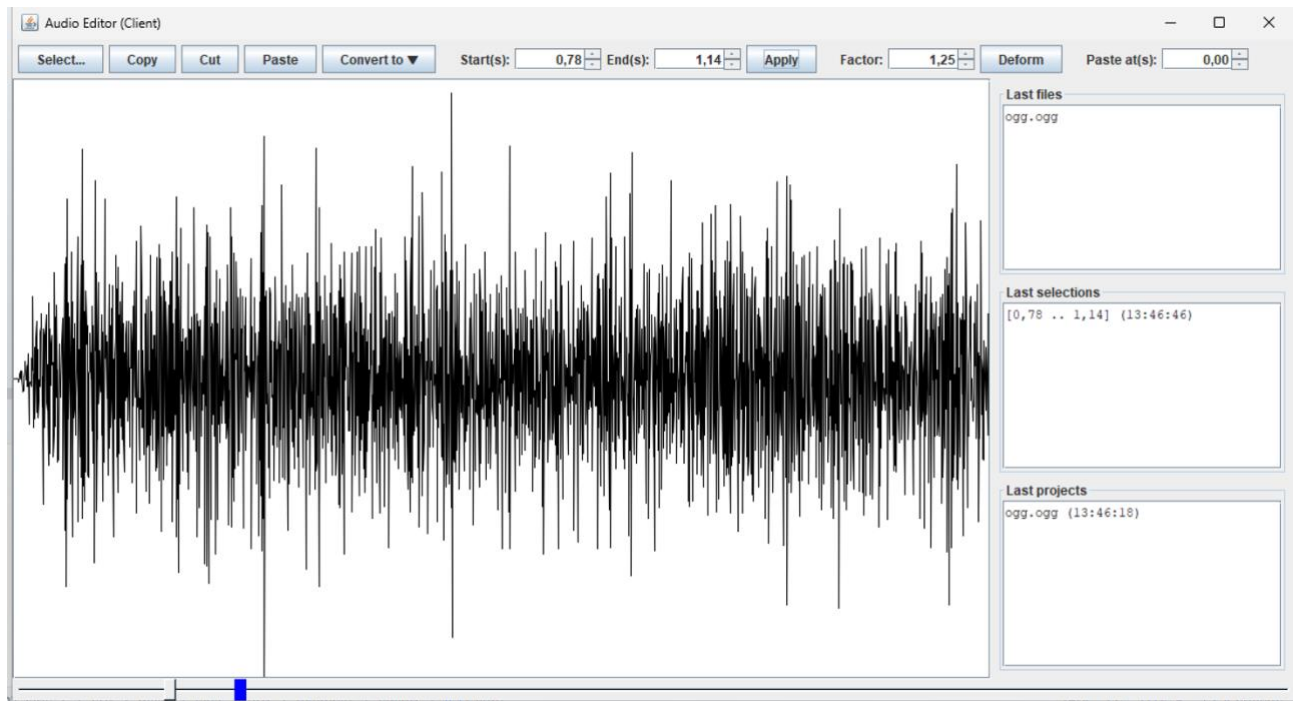


Рис. 3 - Головне вікно застосунку

Головне вікно містить у собі кілька різних компонентів, що разом утворюють цілісну структуру інтерфейсу. У верхній частині розташована панель інструментів із кнопками *Select*, *Copy*, *Cut*, *Paste*, *Convert to*, *Apply* та *Deform*, які дозволяють користувачеві виконувати різні операції з аудіофайлом. Поруч розміщені текстові поля введення (*Start(s)*, *End(s)*, *Factor*, *Paste at(s)*) для задання параметрів редагування. У центрі вікна розташовано основну область відображення звукової хвилі, яка представляє графічне подання аудіосигналу, з можливістю вибору певного фрагмента для редагування. У правій частині вікна знаходяться додаткові панелі - *Last files*, *Last selections* та *Last projects*, де відображається історія нещодавніх дій користувача.

Кожен із цих елементів - кнопки, панелі, поля введення, область візуалізації є окремим компонентом інтерфейсу. Водночас усі вони входять до складу більшого об'єкта - головного вікна програми, що виступає контейнером (*Composite*). Завдяки шаблону *Composite*, усі елементи інтерфейсу можуть оброблятися однаково, як єдина структура. Це дозволяє програмі гнучко

керувати як окремими елементами, так і всім вікном цілком, забезпечуючи узгоджену роботу всіх

Розглянемо кнопку в застосунку, яка містить випадаюче меню (Рис. 4 – Випадаюче меню). При натисканні на *"Convert to"* відкривається список підпунктів із можливими варіантами форматів для конвертації файлів: mp3, ogg, flac.



Рис. 4 – Випадаюче меню

Ці підпункти є окремими елементами, що можуть бути об'єктами в рамках інтерфейсу користувача. Застосовуючи шаблон *Composite*, кожен з них обробляється однаково, як частина загальної структури меню. Користувач не повинен турбуватися про те, чи вибирає він один формат, чи декілька підпунктів - для нього це виглядає як єдина структура, де всі елементи підпорядковуються одній логіці взаємодії.

Таким чином, весь процес вибору формату для конвертації обробляється через один спільний інтерфейс, що полегшує роботу з інтерфейсом і дозволяє розширювати меню без значних змін у загальній структурі програми.

Код системи

Лістинг 1- UIComponent

```
package org.example.composite;

public interface UIComponent {
```

```

        void operation();
    }

```

Лістинг 2- UIContainer

```

package org.example.composite;

import java.util.ArrayList;
import java.util.List;

public class UIContainer implements UIComponent {
    private List<UIComponent> children = new ArrayList<>();

    public void add(UIComponent component) {
        children.add(component);
    }

    public void remove(UIComponent component) {
        children.remove(component);
    }

    @Override
    public void operation() {
        for (UIComponent child : children) {
            child.operation();
        }
    }
}

```

Лістинг 3- UIElement

```

package org.example.composite;

import javax.swing.*;

public class UIElement implements UIComponent {
    private JComponent component;

    public UIElement(JComponent component) {
        this.component = component;
    }

    @Override
    public void operation() {
        component.repaint();
    }
}

```

Лістинг 4- AudioEditorUI (використання шаблону)

```

package org.example.swing;

import org.example.database.DatabaseInitializer;
import org.example.logs.Logger;
import org.example.composite.*;

import javax.swing.*;
import java.awt.*;

public class AudioEditorUI {
    public static void main(String[] args) {
        DatabaseInitializer.initializeDatabase();
    }
}

```

```

JFrame frame = new JFrame("Audio Editor");
frame.setSize(600, 400);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setLayout(null);

JButton loadFileButton = new JButton("Load Audio File");
loadFileButton.setBounds(50, 50, 180, 30);

JLabel fileLabel = new JLabel("No file selected");
fileLabel.setBounds(250, 50, 300, 30);

JButton convertToMp3Button = new JButton("Convert to MP3");
convertToMp3Button.setBounds(50, 100, 180, 30);

JButton convertToOggButton = new JButton("Convert to OGG");
convertToOggButton.setBounds(50, 150, 180, 30);

JButton convertToFlacButton = new JButton("Convert to FLAC");
convertToFlacButton.setBounds(50, 200, 180, 30);

JPanel waveformPanel = new JPanel();
waveformPanel.setBounds(50, 250, 500, 100);
waveformPanel.setBorder(BorderFactory.createLineBorder(Color.BLACK));

Logger logger = new Logger();

UIContainer mainContainer = new UIContainer();

UIContainer buttonPanel = new UIContainer();
buttonPanel.add(new UIElement(loadFileButton));
buttonPanel.add(new UIElement(convertToMp3Button));
buttonPanel.add(new UIElement(convertToOggButton));
buttonPanel.add(new UIElement(convertToFlacButton));

mainContainer.add(buttonPanel);
mainContainer.add(new UIElement(fileLabel));
mainContainer.add(new UIElement(waveformPanel));

frame.add(loadFileButton);
frame.add(fileLabel);
frame.add(convertToMp3Button);
frame.add(convertToOggButton);
frame.add(convertToFlacButton);
frame.add(waveformPanel);

AudioEditorMediator mediator = new AudioEditorMediator(
    loadFileButton, convertToMp3Button, convertToOggButton,
    convertToFlacButton,
    fileLabel, waveformPanel, logger);

mainContainer.operation();

frame.setVisible(true);
}
}

```

Висновок

У ході виконання даної лабораторної роботи було розглянуто та проаналізовано низку шаблонів проектування, що використовуються для взаємодії з базами даних і об'єктами - Active Record, Table Data Gateway, Data Mapper, Composite, Flyweight, Interpreter і Visitor. Теоретичне опрацювання цих шаблонів дало змогу зрозуміти їхні принципи побудови, переваги, недоліки та практичні сценарії застосування в сучасних програмних системах.

Особливу увагу було приділено шаблону *Composite* (Композиція), адже саме він став основою реалізації структури користувацького інтерфейсу у проєкті аудіоредактора.

У процесі створення графічного застосунку для редагування звуку постала потреба в гнучкій структурі, де всі елементи інтерфейсу - кнопки, панелі, поля введення, області візуалізації могли б оброблятися однаково, незалежно від їхньої складності або вкладеності. Саме цю задачу ідеально вирішує шаблон *Composite*, який дозволяє працювати з окремими об'єктами (листами) і групами об'єктів (контейнерами) через спільний інтерфейс.

Composite було обрано з низки причин:

1. Він забезпечує ієрархічну організацію компонентів, дозволяючи розглядати головне вікно, панель інструментів, окремі кнопки та поля як частини одного цілого.
2. Дає змогу керувати групами елементів як єдиним об'єктом. Наприклад, виклик певної операції в головному контейнері може автоматично застосовуватись до всіх дочірніх компонентів.
3. Спрощує розширення системи. У майбутньому можна легко додати нові типи UI-елементів або складніші контейнери, не змінюючи базову логіку.
4. Покращує структурованість коду, що особливо важливо в проєктах із великою кількістю графічних елементів і взаємодій між ними.

У проєкті головне вікно програми виступає контейнером (*Composite*), який містить інші компоненти - *UIElement* (наприклад, кнопки, текстові поля, області відображення сигналу). Клас *UIContainer* відповідає за зберігання та управління цими елементами через методи *add()* і *remove()*. Коли користувач виконує певну дію - наприклад, натискає кнопку - викликається метод *operation()*, який може передавати цю дію всім вкладеним компонентам. Таким чином, головне вікно, панель інструментів, блок візуалізації хвилі та інформаційні панелі (*Last files*, *Last selections*, *Last projects*) об'єднані у спільну структуру, яку *Composite* дозволяє керувати та узгоджено обробляти. Це зробило можливим створення гнучкого інтерфейсу, де будь-який елемент може бути доданий, змінений або видалений без порушення загальної архітектури програми.

У результаті роботи було реалізовано архітектуру, у якій усі графічні компоненти програми взаємодіють як частини єдиної структури. Це значно спростило процес оновлення інтерфейсу, управління подіями та реалізацію нових функцій.

Завдяки *Composite* проєкт став:

- масштабованим, оскільки легко додавати нові елементи UI;
- гнучким, адже будь-яку дію можна застосовувати як до окремого елемента, так і до всієї групи;
- структурно зрозумілим, бо вся логіка організована через єдиний інтерфейс *UIComponent*.

Використання шаблону *Composite* у розробці аудіоредактора продемонструвало його ефективність для створення складних ієрархічних систем, у яких окремі об'єкти й їх комбінації повинні оброблятися узгоджено. Він дозволив реалізувати логічно цілісну структуру головного вікна, у якій усі компоненти - від кнопок до панелей - взаємодіють як єдиний механізм.

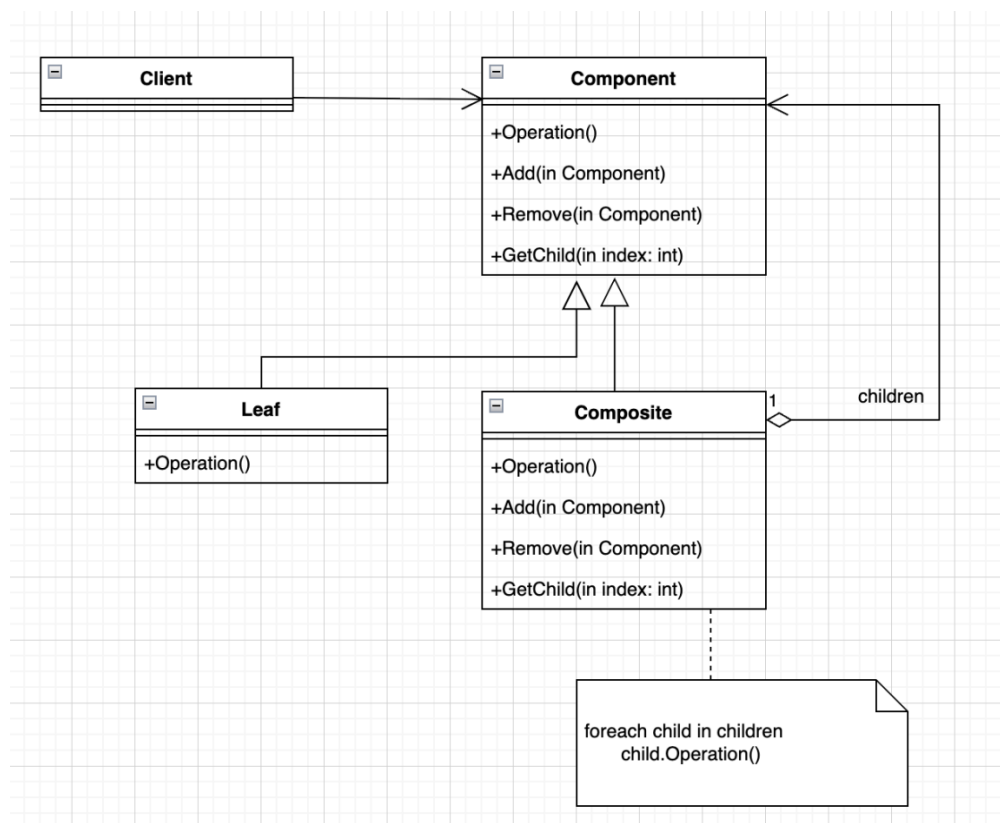
Отже, *Composite* у даному проєкті виконав ключову роль у побудові архітектури користувацького інтерфейсу, забезпечивши гнучкість, зручність керування та можливість подальшого розширення системи без ускладнення коду.

Відповіді на теоретичні питання

1. Яке призначення шаблону «Композит»?

Шаблон «Композит» (*Composite*) призначений для того, щоб об'єднувати об'єкти у деревоподібні структури для представлення ієрархії «ціле–частина». Він дозволяє клієнтам працювати з окремими об'єктами та групами об'єктів однаково.

2. Нарисуйте структуру шаблону «Композит».



3. Які класи входять в шаблон «Композит», та яка між ними взаємодія?

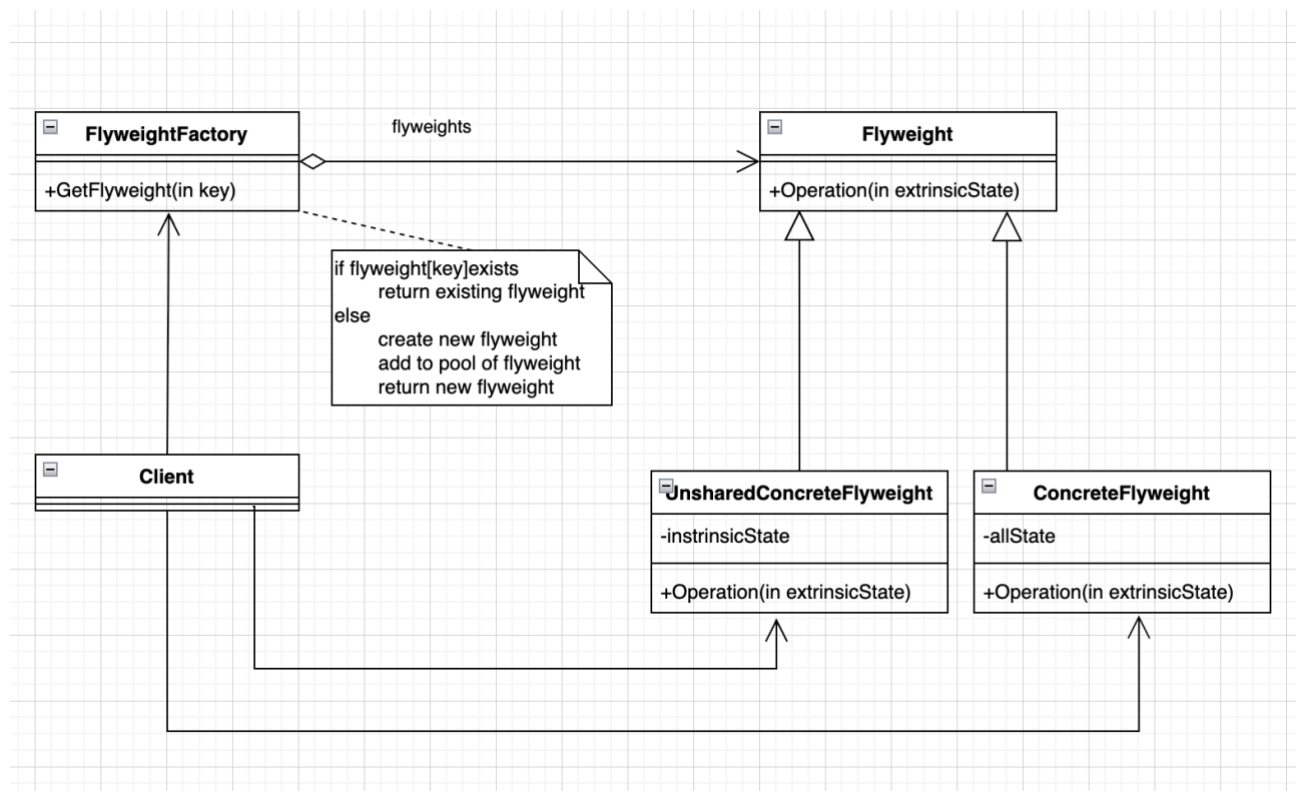
- Component - абстрактний клас або інтерфейс, що визначає загальні операції для всіх елементів дерева.

- Leaf - представник "листка" дерева, який не має дочірніх елементів і реалізує операції, визначені в Component.
- Composite - вузол, що містить інші об'єкти (Leaf або Composite) і реалізує ті ж операції, делегуючи виклики дочірнім елементам.
- Client - працює з об'єктами через інтерфейс Component, не розрізняючи листки та композиції.

4. Яке призначення шаблону «Легковаговик»?

Шаблон «Легковаговик» (*Flyweight*) використовується для оптимізації використання пам'яті, коли програма має велику кількість схожих об'єктів. Ідея полягає в тому, щоб винести спільні дані в окремі об'єкти, які можна розділяти між кількома екземплярами, замість створення копій.

5. Нарисуйте структуру шаблону «Легковаговик».



6. Які класи входять в шаблон «Легковаговик», та яка між ними взаємодія?

- Flyweight - інтерфейс або абстрактний клас, який визначає операції для легковагових об'єктів.

- ConcreteFlyweight - реалізує інтерфейс Flyweight і містить внутрішній (спільний) стан, який можна використовувати повторно.
- UnsharedConcreteFlyweight - об'єкти, що не розділяють спільний стан (використовуються рідше).
- FlyweightFactory - створює та керує об'єктами Flyweight, забезпечуючи їх повторне використання.
- Client - викликає об'єкти Flyweight та зберігає зовнішній (унікальний) стан.

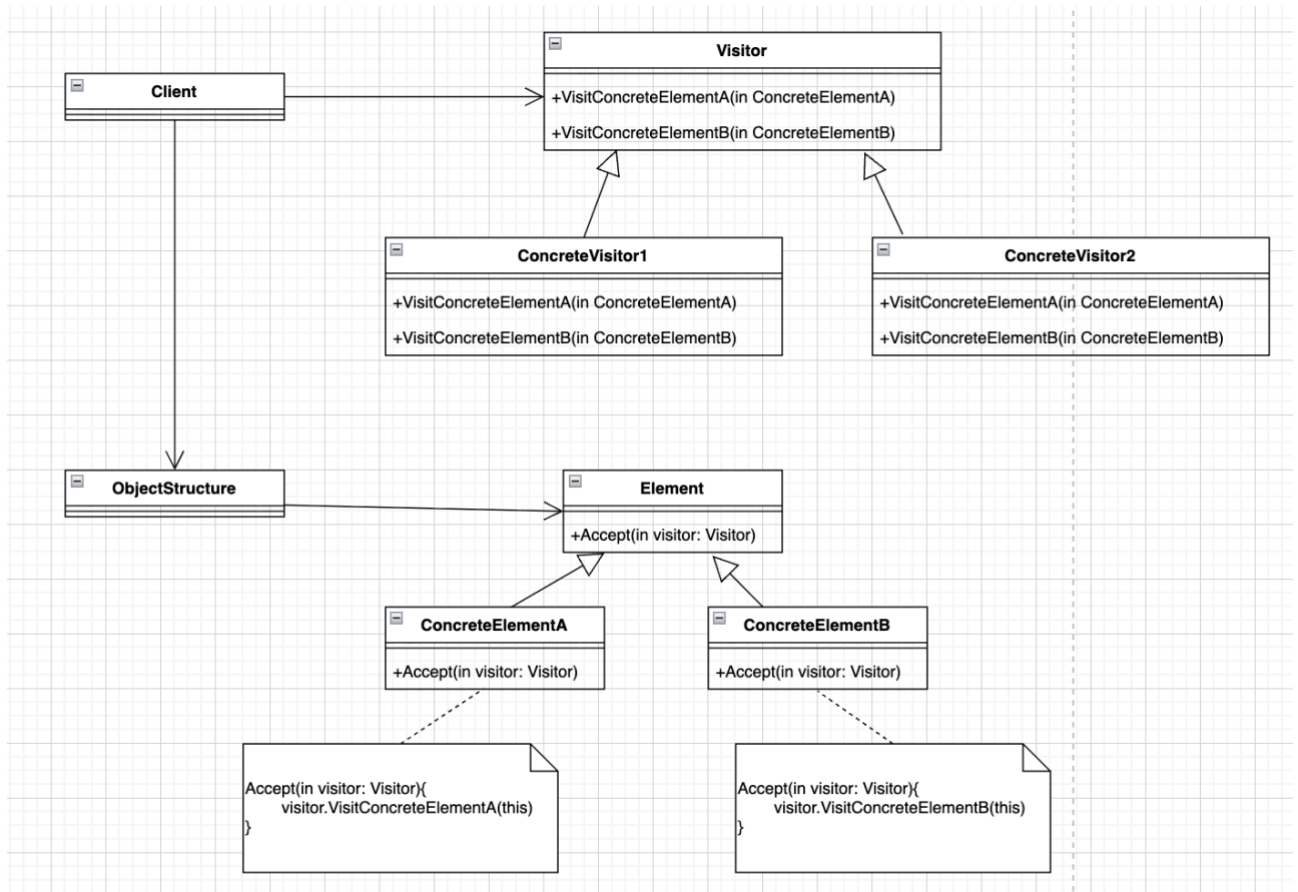
7. Яке призначення шаблону «Інтерпретатор»?

Шаблон «Інтерпретатор» (*Interpreter*) призначений для опису граматики мови та інтерпретації речень цієї мови. Він визначає структуру класів, де кожен клас представляє певне правило граматики, що дозволяє будувати та обчислювати вирази.

8. Яке призначення шаблону «Відвідувач»?

Шаблон «Відвідувач» (*Visitor*) дозволяє додавати нові операції до об'єктів, не змінюючи їх класи. Він відокремлює алгоритм від структури об'єктів, по яких цей алгоритм виконується. Це корисно, коли потрібно виконати різні дії над об'єктами складної структури.

9. Нарисуйте структуру шаблону «Відвідувач».



10. Які класи входять в шаблон «Відвідувач», та яка між ними взаємодія?

- Visitor - інтерфейс, що оголошує методи відвідування для кожного типу елементів структури.
- ConcreteVisitor - реалізація Visitor, що визначає конкретні дії при відвідуванні кожного елемента.
- Element - інтерфейс або абстрактний клас, який приймає відвідувача через метод accept().
- ConcreteElement - конкретні елементи структури, які реалізують метод accept(visitor), викликаючи відповідний метод у відвідувача.
- ObjectStructure - колекція елементів, через яку проходить відвідувач.